はてなブログ　メモ帳

『生成AIと作るローカルLLMサーバ奮闘記』

～Webサーバ構築（第八節　脆弱性診断①）編～

本書について

本書は、はてなブログ『生成AIと作るローカルLLMサーバ奮闘記』の記事を執筆するにあたり、筆者の考えを整理するため作成したドラフト（下書き）です。

同時に、本書をAIに読み込ませることで、実験のコンテキストやコンセプトを伝える役割を持つ資料でもあります。

なお、ブログ上に記載するには情報量が多過ぎるため、ブログ記事とは別に本書をGitHubリポジトリ（https://github.com/hkosho/pimientito_blog_docs）上で公開いたします。

ブログ本編と合わせてご覧いただき、実験の意図がより深くお伝えできれば幸いです。

2026/03/17

pimientito

課題認識

AIは、一度に大量の情報を出力する。生成するプログラムコード（以後、コード）も同様である。一般的な文章とは異なり、コードの構造は複雑で、内容の理解は容易ではない。

完成品に近い状態のコードには、設計書のような補助的な資料は無く、私たちは、白紙の状態から全体を読み解かなければならない。

このような状況では、従来の手法（ホワイトボックス／ブラックボックス）を用いた検証にも限界がある。そこで、複数のAIを組み合わせ、相互にコードをデバッグさせることで、初歩的なミスを事前に発見できれば、人間側の確認負担が減るのではないかと考えた。

今回の実験では、AIとの協働開発において、特に検証分野について境界線を探る。

今回の目標

　現在、自社のWebサイトリニューアルをAI（ChatGPT / Claude / Gemini）と協働で開発している。開発サーバのDockerコンテナ上には、Webサイトが構築されている。

　今回は、従来のシステムテストにAIを組み込み、AI自らバグや脆弱性を発見できるかを見極める。

　基本的な検証方法は、従来のスタイルを踏襲する。具体的には、IPA（独立行政法人　情報処理推進機構）の資料を読み込ませ、IPA推奨の脆弱性対策に沿った検証方針を計画させる。

　また、検証ツールは、OSS（オープンソースソフトウェア）に準拠した無償版を使用する。

検証フロー概要

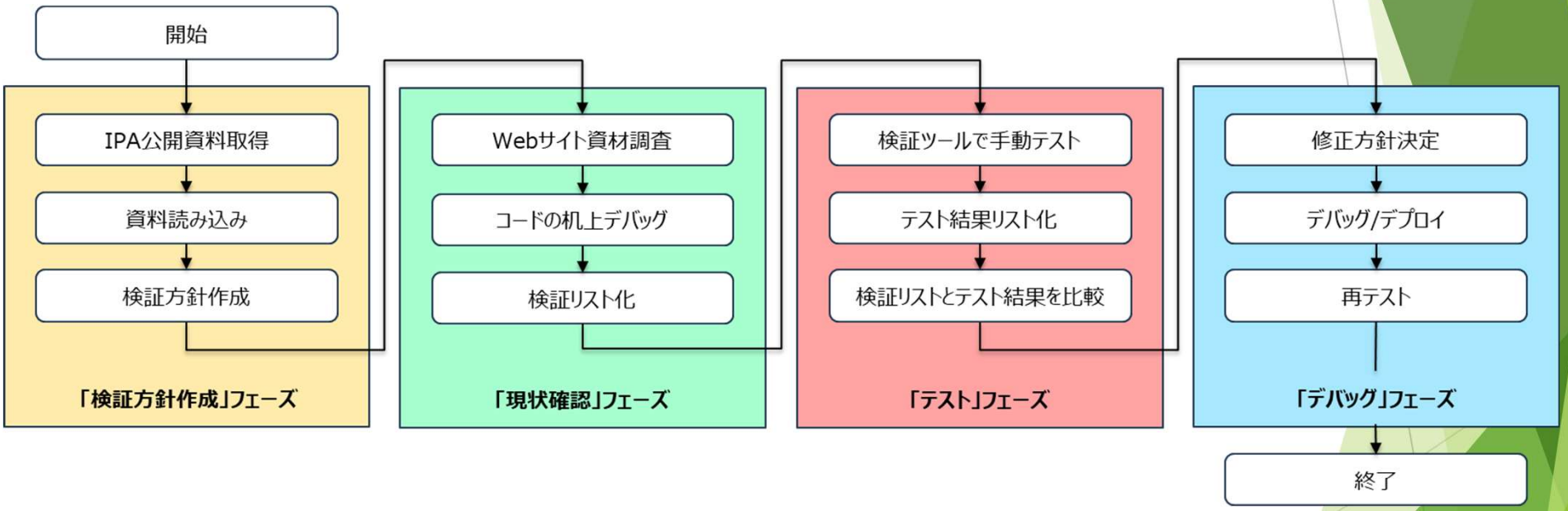
1. IPA（※1）公開「安全なウェブサイトの作り方」（※2）から検証方針を作成
2. Gitリポジトリ上の自社Webサイトのプロジェクト資材を抽出
3. 生成AI（ChatGPT/Claude/Gemini）による調査
4. 脆弱性のあるファイルやコードの机上デバッグを実施
5. 検証リスト作成
6. 人間がツールを使用してプロジェクト資材を検証（※3）
7. 検証リストとテスト結果を比較
8. 修正方針を決定
9. デバッグ/デプロイ

※1 IPA（独立行政法人　情報処理推進機構）

※2 「安全なウェブサイトの作り方」（https://www.ipa.go.jp/security/vuln/websecurity/about.html）

※3 IPA公開「安全なウェブサイトの作り方　改訂第7版」（チェックリスト）等を参考に人間による実施

検証フロー図


開始
IPA公開資料取得
資料読み込み
検証方針作成
「検証方針作成」フェーズ
Webサイト資材調査
コードの机上デバッグ
検証リスト化
「現状確認」フェーズ
検証ツールで手動テスト
テスト結果リスト化
検証リストとテスト結果を比較
「テスト」フェーズ
修正方針決定
デバッグ/デプロイ
再テスト
「デバッグ」フェーズ
終了

1.「検証方針作成」フェーズ

- **概要**

　IPA（独立行政法人　情報処理推進機構）が公開する「**安全なウェブサイトの作り方**」等を、生成AI（ChatGPT／Claude／Gemini）に読み込ませ、現在、開発中のWebサイトの検証方針を作成する。

- **具体例**
  - Webサイトに必要なディレクトリやファイルの基本構成を把握
  - 開発言語やフレームワークの特性を把握
  - 既知の検証項目の洗い出し
- **期待値**
  - 生成AIがWebサイト構築の基礎を理解し、評価基準を定義できること
  - 開発中のWebサイトに適した検証方針が作成できること
  - 各AIが提案する方針に癖や違いを発見できること（保守的、網羅的など）

2. 「現状確認」フェーズ

- **概要**

　Gitリポジトリ上のプロジェクト資材を、AIが生成したコードの完成品として扱うのではなく、AIからの提案と捉え「**技術的検証**」「**意味論的照合**」「**持続性と安全性**」の視点からプロジェクト資材を検証する。

- **具体例**
  - 技術的検証：動作することを機械的に確認（動作検証等）
  - 意味論的照合：要件と実体に乖離がないことを確認（机上デバッグ等）
  - 持続性と安全性：運用時に負債を抱えないことを確認（メンテナンス性等）
- **期待値**
  - 技術的検証：構文エラーや型定義の矛盾を検出
  - 意味論的照合：異なるAI同士のインタビュー形式でコードを机上デバッグ
  - 持続性と安全性：コーディングのハルシネーションや規約違反の検出

3.「テスト」フェーズ

- **概要**

　従来の人によるプログラムテスト同様に、静的テスト（机上デバッグ等）、動的テスト（検証ツール等）を実施。

- **具体例**
  - IPAが公開するチェックリストを使用してコードを静的に検証
  - ツールを使用してDockerコンテナ上のWebサイトを動的に検証
- **期待値**
  - AI同士の机上デバッグでは発見できなかった不具合を発見
  - AIが指摘していない脆弱性を検証ツールで発見
- **懸念点**
  - オンプレミス環境とDockerコンテナ環境のテスト手法の相違点の確認

代表的な脆弱性診断ツール一覧

診断カテゴリ	名称	主なツール名	カテゴリの説明	ツールの役割
DAST Dynamic Application Security Testing	動的アプリケーションセキュリティテスト	OWASP ZAP	・動作中のアプリケーションに外部からリクエストを送信し脆弱性を検出 ・実行時に現れる問題を検出	・XSS、CSRF、ディレクトリトラバーサル等を外部から自動検査
	SQLインジェクション特化	sqlmap	・動作中のアプリケーションに外部からリクエストを送信し脆弱性を検出 ・SQLインジェクションに特化した動的検査	・HTTPリクエストのパラメータに対しSQLインジェクションの危険性を検出 ・お問い合わせフォーム等の入力項目が対象。
Network Scanning （Port Scanning）	ネットワークスキャン （ポートスキャン）	Nmap	・ネットワーク層の意図しないポート公開や不要なサービス稼働を検出	・公開ポート、稼働サービス、TLS/SSL設定を検査
SAST Static Application Security Testing	静的アプリケーションセキュリティテスト	Semgrep	・アプリケーション未実行でソースコードを解析し脆弱性を検出 ・コードレベルの問題を検出	・TypeScript／JavaScriptのソースコードを静的解析 ・XSS、インジェクション、危険なAPI使用等のパターン検知
SCA Software Composition Analysis	ソフトウェア構成分析	Trivy	・プロジェクトが依存しているサードパーティのライブラリやパッケージの脆弱性を検査 ・依存パッケージの問題を検出	・npm／pnpmパッケージの既知の脆弱性検出 ・Dockerイメージのスキャン、Linuxパッケージのバージョン検証
VA Vulnerability Assessment	脆弱性評価	OpenVAS	・既知のセキュリティ脆弱性をCVEデータベース（※）と照合して包括的に診断 ※ Common Vulnerabilities and Exposures 米国MITRE社が採番／管理する既知のセキュリティ脆弱性IDと脆弱性情報を公開するデータベース	・OS・ミドルウェア・ネットワーク機器の既知の脆弱性を網羅的にスキャン

※ 補足：SAST・SCA・DASTは補完関係にあり、3つを組み合わせることで診断のカバー範囲が広がる。

4. 「デバッグ」フェーズ

- **概要**

　AIによる静的な検証と人間による動作検証の結果をまとめ比較する。AIが何を予測できて、何が考慮できなかったのかを明らかにする。

　結果を基にAIによる修正を行い、IPA推奨の対策が取れていることを再検証する。

- **具体例**
  - AIによる評価と人による検証の結果を比較
  - Webコンテンツ、サーバ、ネットワーク別に成功例と失敗例をまとめる
- **期待値**
  - AIの推測と実状の乖離が明らかになる
  - インターネット上のサンプルコードとの類似点の発見
- **懸念点**
  - AIの先回りによりデバッグ中にリグレッション（デグレード／退行）が発生

期待する結論

今回の実験では、AIの検証が外れるネガティブな結果から、より有意義な知見が得られるのではないかと予測している。

AIは、もっともらしい出力をするよう設計されているが、必ずしも実環境で正しく動作するとは限らない。

プログラミングは、修正を繰り返しながら構築されるものであるが、AIのアウトプットからは、一度の生成で完結しようとする危うさが見受けられる。

ブラックボックス化された生成AIの不得意な分野や傾向を掴むことは、より安全で安定したシステム開発の一助になると考えている。

参考資料（2026/03/17時点）

- 「安全なウェブサイトの作り方」（IPA 独立行政法人　情報処理推進機構）
- 「セキュリティ実装　チェックリスト」（IPA 独立行政法人　情報処理推進機構）
  https://www.ipa.go.jp/security/vuln/websecurity/about.html
- OWASP ZAP
  https://www.zaproxy.org/
- Sqlmap
  https://sqlmap.org/
- Nmap
  https://nmap.org/
- Semgrep
  https://semgrep.dev/
- Trivy
  https://trivy.dev/
- OpenVAS
  https://openvas.org/
- 「体系的に学ぶ　安全なWebアプリケーションの作り方」徳丸浩著（SB Creative）

ご清聴、誠にありがとうございました。